

Esercizio 3 in Python — Cassetta degli attrezzi completa

Tutto ciò che serve (moduli, funzioni, idiomi) per risolvere **tutti** gli Esercizi 3 delle prove pratiche, più la mappatura prova-per-prova.

Regola d'oro della prova: **mai** `os.system`, `os.popen`, `os.spawn*`. Per lanciare comandi esterni si usa **solo** `subprocess` (e mai con `shell=True`, che equivale a `sh -c`). Quasi tutti gli Esercizi 3 però si risolvono **senza** lanciare comandi.

1. Lo scheletro di partenza di ogni script

```
#!/usr/bin/env python3
import sys, os

def main():
    args = sys.argv[1:]          # parametri (argv[0] è il nome del programma)
    # ... logica ...

if __name__ == "__main__":
    main()
```

- `sys.argv` → lista dei parametri; `sys.argv[0]` è il nome con cui è stato invocato (serve in 2025.05.29: comportamento diverso se chiamato `cksymlink` o `cklink`).
- `sys.exit(codice)` → uscita con valore di ritorno.
- Argomento opzionale "directory o cwd": `d = args[0] if args else "."`.
- Ricorda `chmod +x script` e lo shebang `#!/usr/bin/env python3` (teoria: appunti esercitazioni, sezione "Shell scripting / Invocazione").

2. Visitare un albero del file system (il cuore di ~20 prove)

`os.walk` — il metodo principale

```
for dirpath, dirnames, filenames in os.walk(root):
    for name in filenames:
        full = os.path.join(dirpath, name)
        ...
# per visita post-order (prima i figli, poi la radice): os.walk(root, topdown=False)
```

- `dirpath` è il percorso della cartella corrente, `dirnames / filenames` i nomi (non i path completi) → ricomponi con `os.path.join`.
- `os.walk(root, topdown=False)` visita **dal basso** → utile per 2025.01.21 (file più profondi prima) e per cancellazioni ricorsive.
- `os.walk(..., followlinks=False)` (default) **non segue** i symlink di directory: fondamentale per non entrare in loop e per gli esercizi sui link.
- **Attenzione:** `os.walk` di default elenca solo file e dir; i symlink a file compaiono in `filenames`, i symlink a directory in `dirnames`.

`os.scandir` — più efficiente, dà già il tipo

```
with os.scandir(path) as it:
    for entry in it:
        entry.name, entry.path
        entry.is_file(follow_symlinks=False)
        entry.is_dir(follow_symlinks=False)
        entry.is_symlink()
        entry.stat(follow_symlinks=False)    # come lstat
```

Ricorsione manuale con `os.listdir`

```
def walk(path):
    for name in os.listdir(path):
        full = os.path.join(path, name)
        st = os.lstat(full)          # lstat: NON segue il link
        if stat.S_ISDIR(st.st_mode):
            walk(full)
        else:
            ...
```

`pathlib` (alternativa orientata agli oggetti)

```

from pathlib import Path
root = Path(".")
for p in root.rglob("*"):          # ricorsivo, tutti i file/dir
    p.name, p.suffix, p.is_symlink(), p.stat().st_size
for p in root.glob("*.c"):         # solo nel livello corrente
    ...

```

3. Metadati dei file: `os.stat` / `os.lstat` e il modulo `stat`

```

import os, stat
st = os.stat(path)                # SEGUE i symlink (come stat())
st = os.lstat(path)               # NON segue i symlink (come lstat())

st.st_size      # dimensione in byte          -> 2025.02.13 (stessa ampiezza)
st.st_ino       # numero di inode              -> 2023.05.24, 2023.09.14, 2025.09.04
st.st_dev       # device (inode unico = (st_dev, st_ino))
st.st_nlink     # numero di hard link
st.st_mtime     # ultima modifica (float, secondi) -> ordinamenti per data
st.st_mode      # tipo + permessi
st.st_uid       # proprietario                 -> /proc, processi utente

# Tipo di file
stat.S_ISREG(st.st_mode) # file regolare
stat.S_ISDIR(st.st_mode) # directory
stat.S_ISLNK(st.st_mode) # link simbolico (usare con lstat!)

# Permessi
perm = stat.S_IMODE(st.st_mode) # solo i bit di permesso (es. 0o644)
stat.filemode(st.st_mode)       # stringa stile ls: '-rw-r--r--' -> 2024.06.24
oct(perm)                       # '0o644'

```

Teoria collegata: dispensa §3.2 (`struct stat` , `st_mode & 07777` , macro `S_IF*`) e §3.3 (`stat` vs `lstat`). `stat.filemode` riproduce esattamente la prima colonna di `ls -l` .

4. Path: il modulo `os.path` (e `pathlib`)

```

os.path.join(a, b)          # concatena pezzi di path
os.path.basename(p)         # nome file
os.path.dirname(p)          # cartella
os.path.splitext(p)         # ('nome', '.ext') -> suffisso (2020.02.21)
os.path.abspath(p)          # path assoluto
os.path.realpath(p)         # path assoluto CANONICO (risolve i symlink) = realpath(3)
os.path.relpath(p, start)   # path relativo rispetto a start -> 2024.07.23 (symlink relativi)
os.path.isabs(p)            # è assoluto? -> 2023.06.14 (relativi vs assoluti)
os.path.exists(p)           # esiste? (segue symlink; lexists NON segue)
os.path.lexists(p)          # esiste il link anche se rotto
os.path.commonpath([a, b])  # prefisso comune -> 2024.09.11 (interni vs esterni)
os.path.samefile(a, b)      # True se stesso inode (hard link) -> 2025.02.13, 2025.05.29

```

5. Link simbolici e fisici

```

os.readlink(link)           # restituisce il TARGET testuale del symlink
os.symlink(target, linkpath) # crea un symlink
os.link(src, dst)           # crea un HARD link
os.unlink(path) / os.remove(path) # cancella un nome (file o symlink)
os.rename(src, dst)         # rinomina/sposta in modo ATOMICO -> 2024.07.23

```

Idiomi chiave: - **Symlink che puntano allo stesso file** (2023.05.24, 2023.09.14): per ogni symlink calcola `os.path.realpath(link)` (o `os.stat(link).st_ino` che segue il link) e raggruppa per inode/target. - **Hard link di f** (2025.05.29): file con `os.lstat(x).st_ino == os.lstat(f).st_ino` e stesso `st_dev` , oppure `os.path.samefile(x, f)` . - **Symlink che puntano a f** (2025.05.29): per ogni symlink, `os.path.realpath(link) == os.path.realpath(f)` . - **Symlink relativi vs assoluti** (2023.06.14): `os.path.isabs(os.readlink(link))` . - **Interno vs esterno** (2024.09.11): risolvi il target ad assoluto e controlla se sta dentro la radice → `os.path.realpath(target).startswith(os.path.realpath(root))` (meglio con `os.path.commonpath`).

6. Leggere/scrivere file e confrontare contenuti

```
# Testo
with open(path, encoding="utf-8") as f:
    for line in f: ...
    data = f.read()

# Binario (per confronti byte a byte / file non testuali)
with open(path, "rb") as f:
    blob = f.read()

# Confronto contenuti
import filecmp
filecmp.cmp(a, b, shallow=False) # True se contenuto identico -> 2025.07.23, 2023.01.23

# Scrivere/creare
with open(out, "w", encoding="utf-8") as f: f.write(testo)
with open(out, "ab") as f: f.write(b"...") # append binario -> 2023.01.19 (concatena)
```

- **Pattern dentro i file** (2022.01.18, 2022.02.17, 2026.02.09): apri ogni file regolare, cerca la stringa. Per evitare crash su file binari usa `open(p, "rb")` e cerca i byte (`pattern.encode() in blob`) oppure `open(p, errors="ignore")` .
- **Conteggio caratteri per riga** (2023.02.16 `ccpl`): per ogni file leggi le righe, accumula in un dict `riga_n -> totale_caratteri` .
- **Contenuto UTF-8 → ASCII con '?'** (2026.01.12): vedi §10.

7. Ordinare (chiave `key=`)

```
sorted(lista) # alfabetico
sorted(files, key=lambda p: os.stat(p).st_mtime) # per data, crescente
sorted(files, key=lambda p: os.stat(p).st_mtime, reverse=True) # decrescente
sorted(files, key=lambda p: os.path.splitext(p)[1]) # per suffisso (2020.02.21)
sorted(files, key=lambda p: os.lstat(p).st_ino) # per inode (2025.09.04)
```

- **Più recente/più vecchio** (2021.07.15): `max(files, key=mtime)` e `min(files, key=mtime)` .
- **Dal più recente al più remoto** (2022.01.18/02.17): `reverse=True` .
- **Dal più vecchio al più recente** (2026.02.09): `reverse=False` .
- **Per profondità** (2025.01.21): chiave = numero di `/` nel path relativo (livello); ordina per (`-livello, nome`) così i più profondi vengono prima e a parità di livello l'ordine è alfabetico. Oppure `os.walk(topdown=False)` .

8. Lanciare comandi esterni: `subprocess` (mai `shell=True`)

```
import subprocess

# Eseguire e aspettare, catturando l'output
r = subprocess.run(["file", path], capture_output=True, text=True)
r.stdout, r.returncode

# Lanciare in parallelo e aspettare tutti (2020.01.17 para.py)
procs = [subprocess.Popen(cmd) for cmd in lista_di_liste]
for p in procs: p.wait()
```

Esercizi che richiedono comandi esterni: - **file** per la categoria (2022.06.22, 2024.02.14): `subprocess.run(["file", "-b", p], capture_output=True, text=True).stdout.strip()` . - **b** toglie il nome dall'output. - **Indentazione C** (2021.06.24): la traccia suggerisce `ex -n '+norm!gg=G' +wq file.c` → `subprocess.run(["ex", "-n", "+norm!gg=G", "+wq", p])` . - **Lancio parallelo** (2020.01.17): split dell'input sui `//` , ogni pezzo diventa una lista di argomenti, `Popen` per ciascuno, poi `wait` su tutti. - **Esegui script ma non binari ELF** (2022.07.22): vedi §9 per riconoscere ELF, poi `subprocess.run([percorso_script])` .

Per "comando con parametri" passa **una lista** (`["ls", "-l"]`), non una stringa: così eviti del tutto la shell.

9. Riconoscere il tipo di un eseguibile (ELF vs script)

Due strade: - **Leggere il magic number** (no comandi esterni): un ELF inizia con i 4 byte `\x7fELF` ; uno script con `#!` .

```
with open(path, "rb") as f:
    head = f.read(4)
is_elf = head[:4] == b"\x7fELF"
with open(path, "rb") as f:
    is_script = f.read(2) == b"#!"
```

- **Usare file** via `subprocess` (vedi §8) e cercare "ELF" nella stringa.
- **È eseguibile?** `os.access(path, os.X_OK)` .

Esercizi: 2022.07.22 (esegui script, salta ELF), 2022.09.07 (`elfsize` : somma `os.stat(p).st_size` di tutti i file il cui magic è `\x7fELF`).

10. Caratteri ASCII / UTF-8 (2023.07.20 e 2026.01.12)

- **Nomi non rappresentabili in ASCII** (2023.07.20):

```
def is_ascii(name):
    try:
        name.encode("ascii"); return True
    except UnicodeEncodeError:
        return False
# elenca file/dir per cui not is_ascii(nome)
```

- **Copia file UTF-8 sostituendo i non-ASCII con '?'** (2026.01.12 `no8cat`):

```
with open(filein, encoding="utf-8") as fi, open(fileout, "w", encoding="ascii") as fo:
    for ch in fi.read():
        fo.write(ch if ord(ch) < 128 else "?")
```

Punto delicato: lavorando per **carattere** (non per byte) ogni emoji o lettera accentata diventa un singolo `?`, come richiede l'esempio (cioè `I♥Y` → `cio? I?Y`).

11. Hashing SHA1 (2024.07.23 t1, 2025.07.23)

```
import hashlib

def shal(path):
    h = hashlib.shal()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(65536), b""):
            h.update(chunk)
    return h.hexdigest()
```

- **Cancellare i file con stessa sha1 presenti in entrambe le dir** (2024.07.23 t1): calcola le sha1 dei file di `a` e di `b`, fai l'intersezione degli hash, poi cancella in entrambe i file il cui hash è nell'intersezione.
- In alternativa al contenuto-uguale si può usare `filecmp.cmp(..., shallow=False)` (2025.07.23 `dremcont`), ma con tanti file l'hash è più comodo per il confronto a coppie.

12. Processi e `/proc` (2022.06.08, 2024.05.30)

Linux espone i processi come directory in `/proc/<pid>/`:

```
for pid in os.listdir("/proc"):
    if not pid.isdigit(): continue
    base = f"/proc/{pid}"
    # filtra per utente: confronta os.stat(base).st_uid con os.getuid()
    # eseguibile reale:
    exe = os.readlink(f"{base}/exe") # 2024.05.30 (raggruppa per questo)
    # riga di comando (argomenti separati da \0):
    with open(f"{base}/cmdline", "rb") as f:
        argv = f.read().split(b"\0")
    # status (chiave: valore), contiene Name e VmSize:
    with open(f"{base}/status") as f:
        info = dict(l.split(":",1) for l in f if ":" in l)
    info["Name"].strip(), info.get("VmSize", "").strip() # 2022.06.08
```

- `os.getuid()` → uid dell'utente corrente (per filtrare "i suoi" processi).
- Gestire `PermissionError` / `FileNotFoundError` con try/except: alcuni `/proc/pid` spariscono mentre li leggi.
- **Raggruppare per eseguibile** (2024.05.30 `lscmd`): `dict exe -> [pid, ...]`.

13. Creare/copiare/spostare (modulo `shutil` e `os`)

```
import shutil
shutil.copy2(src, dst) # copia file con metadati
shutil.copytree(src, dst) # copia un intero albero
os.makedirs(path, exist_ok=True) # crea dir (e genitori)
os.mkdir(path, mode)
```

- **Merge di due alberi in un terzo** (2023.01.19): `os.walk` su entrambi; se il path relativo è una dir creala, se è un file copialo, e se esiste già **appendi** il contenuto (`open(dst, "ab")`).
- **diffdir a b newa newb** (2023.01.23): nomi comuni = intersezione di `set(os.listdir(a)) & set(os.listdir(b))`; `copy2` di ciascuno in `newa / newb`.
- **permdir** (2024.06.24): per ogni file calcola `stat.filemode`, crea una dir con quel nome (`makedirs(exist_ok=True)`) e dentro un

`os.symlink` al file.

- **dir ... + symlink relativi** (2024.07.23 t2): `os.makedirs("../", exist_ok=True)`, `os.rename` del file nella sottodir, poi `os.symlink(os.path.relpath(nuova, "."), nome)`.

14. Confronto di alberi (2024.01.19 `tcmp`)

Riduci ogni albero all'insieme dei suoi path **relativi**:

```
def relset(root):
    out = set()
    for dp, dns, fns in os.walk(root):
        for n in dns + fns:
            out.add(os.path.relpath(os.path.join(dp, n), root))
    return out

a, b = relset(dir1), relset(dir2)
a & b      # comuni (nessuna opzione)
a - b      # solo nel primo (-1)
b - a      # solo nel secondo (-2)
```

`set` (unione `|`, intersezione `&`, differenza `-`) è lo strumento giusto anche per 2023.01.23 e 2024.07.23.

15. Mappa rapida prova → strumenti

Prova	Cosa fa l'Es.3	Strumenti Python chiave
2020.01.17	lancia comandi separati da <code>//</code> in parallelo, aspetta tutti	<code>subprocess.Popen</code> + <code>wait</code> , split su <code>//</code>
2020.02.21	listato dir ordinato per suffisso	<code>os.listdir</code> , <code>os.path.splitext</code> , <code>sorted(key=)</code>
2021.06.24	corregge indentazione dei <code>.c</code> / <code>.h</code> nel sottoalbero	<code>os.walk</code> , <code>subprocess.run(["ex",...])</code>
2021.07.15	file più recente e più vecchio del sottoalbero	<code>os.walk</code> , <code>st_mtime</code> , <code>max / min</code>
2021.09.16	lista ricorsiva: in quali sottodir compare ogni nome	<code>os.walk</code> , dict <code>nome->[dir]</code> , <code>sorted</code>
2022.01.18	file che contengono un pattern, dal più recente	<code>os.walk</code> , lettura binaria, <code>sorted(reverse=True)</code>
2022.02.17	idem (sottoalbero)	come sopra
2022.06.08	tabella processi utente: nome + VmSize	<code>/proc/*/status</code> , <code>st_uid</code> , <code>os.getuid</code>
2022.06.22	catalogo file per output di <code>file</code>	<code>subprocess.run(["file","-b"])</code> , dict categoria->file
2022.07.22	esegue script ma non binari ELF	magic <code>\x7fELF / #!</code> , <code>subprocess.run</code>
2022.09.07	byte totali degli eseguibili ELF	magic ELF, <code>st_size</code> , somma
2023.01.19	merge di due alberi, concatena omonimi	<code>os.walk</code> , <code>copy2</code> , append <code>"ab"</code>
2023.01.23	<code>difdir</code> : solo nomi comuni nelle due nuove dir	<code>set &</code> , <code>copy2</code> , <code>makedirs</code>
2023.02.16	<code>ccpl</code> : somma caratteri delle righe corrispondenti	lettura righe, dict <code>n->totale</code>
2023.05.24	symlink che indicano lo stesso file (inode)	<code>os.path.realpath</code> , <code>st_ino</code> , raggruppa
2023.06.14	cancella symlink relativi, tieni gli assoluti	<code>os.readlink</code> , <code>os.path.isabs</code> , <code>os.unlink</code>
2023.07.20	nomi non rappresentabili in ASCII	<code>name.encode("ascii")</code> + try/except
2023.09.14	symlink che puntano allo stesso file	<code>os.path.realpath</code> , raggruppa per target/inode
2024.01.19	<code>tcmp</code> : path comuni/-1/-2	<code>os.walk</code> , <code>os.path.relpath</code> , operazioni su <code>set</code>
2024.02.14	<code>catls</code> : catalogo per categoria di <code>file</code>	<code>subprocess.run(["file","-b"])</code> , dict
2024.05.30	<code>lscmd</code> : PID utente per pathname eseguibile	<code>/proc/*/exe</code> , <code>readlink</code> , <code>st_uid</code>
2024.06.24	<code>permdir</code> : dir per stringa permessi con symlink	<code>stat.filemode</code> , <code>makedirs</code> , <code>os.symlink</code>
2024.07.23 t1	cancella file con stessa sha1 in entrambe le dir	<code>hashlib.sha1</code> , <code>set &</code> , <code>os.unlink</code>
2024.07.23 t2	dir <code>...</code> + symlink relativi	<code>makedirs</code> , <code>os.rename</code> , <code>os.path.relpath</code> , <code>os.symlink</code>
2024.09.11	<code>slinout</code> : symlink interni vs esterni	<code>os.readlink</code> , <code>realpath</code> , <code>commonpath</code>
2025.01.21	file ordinati per profondità (più profondi prima)	<code>os.walk(topdown=False)</code> o chiave <code>(-livello,nome)</code>
2025.02.13	file stessa dimensione di f ma non hard link	<code>st_size</code> , <code>os.path.samefile</code>
2025.05.29	<code>cksymlink / cklink</code> secondo <code>argv[0]</code>	<code>sys.argv[0]</code> , <code>realpath</code> / <code>samefile</code>
2025.06.24	aggiunge commenti in testa a <code>.c</code> / <code>.sh</code> / <code>.py</code>	per suffisso, rispetta <code>/* */</code> , <code>#</code> , <code>''' '''</code> , gestisci <code>#!</code>
2025.07.23	<code>dremcont</code> : cancella file con contenuto = f	<code>filecmp.cmp(shallow=False)</code> o sha1, <code>os.unlink</code>
2025.09.04	<code>inotab</code> : inode+path ordinati per inode, profondità	<code>os.walk</code> , <code>st_ino</code> , <code>sorted</code> , conteggio livelli
2026.01.12	<code>no8cat</code> : copia UTF-8 → non-ASCII diventa <code>?</code>	lettura per carattere, <code>ord(ch) < 128</code>
2026.02.09	pattern nei file, dal più vecchio al più recente	come 2022.01.18 ma <code>reverse=False</code>

16. Insidie ricorrenti (perché si perdono punti)

- `stat` vs `lstat` : sui link usa SEMPRE `os.lstat` / `follow_symlinks=False` , o vedrai il bersaglio invece del link.
- `shell=True` / `os.system` : vietati. Passa liste di argomenti a `subprocess` .
- File binari letti come testo:** avvolgi le letture in try/except o apri in `"rb"` quando cerchi pattern/contenuti.
- /proc che cambia:** i processi spariscono mentre leggi → try/except su ogni pid.
- Path relativi vs assoluti:** per i symlink "relativi" (2024.07.23) usa `os.path.relpath` ; per confronti di identità file usa `os.path.realpath` o `os.path.samefile` .
- Symlink rotti:** `os.path.exists` ritorna False; usa `os.path.lexists` per sapere se il link esiste comunque.
- Profondità/ordinamento:** definisci con cura la chiave; per "stesso livello in ordine crescente" usa una tupla `(livello, nome)` come chiave di `sorted` .
- Consegna:** niente commenti nei file (richiesto dal 2025 in poi), rinomina i `.py` , allega gli `sha1sum` .